CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chandubhai S Patel Institute of Technology

Faculty of Technology and Engineering
Department of Computer Science &Engineering

Practical 2

Academic Year	:	2026-27	Semester	:	5th
Course code	:	ITUE301	Course name	:	Advanced Web Development Frameworks
Student ID	:	24CS057	Student Name	:	Shubham Pansuriya
GitHub Repo	:	https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%202			

❖ Problem Definition & Objectives

Course Outcome (CO): CO1 - Implement reactive state management and multi-page client-side navigation using modern web development frameworks.

Objective: To implement reactive state management using useState and multi-page navigation using React Router in the portfolio application without full page reloads.

Problem Statement Requirements:

- Extend the portfolio application from Practical 1 by adding React Router v6.
- Implement a navigation bar with at least 3 routes: Home (/), Projects (/projects), and Contact (/contact).
- Each route must render a distinct component without triggering a full page reload.
- Add at least 2 useState variables used meaningfully: one for toggling UI visibility and one for managing form input on the Contact page.
- The Contact page must include a controlled input that captures and displays user input in real time.

Supplementary Objectives Completed:

- Added a 404 Not Found route (path='*') that renders a custom error component for unknown routes.
- Implemented a dark/light mode toggle using useState that dynamically toggles CSS classes on the root element.
- Stored the contact form input in state and rendered a live character counter below the input.

❖ Architecture & Component Tree

```
App.jsx (wrapped in <BrowserRouter>)
 ├── NavBar.jsx (links to all routes using <NavLink>, dark/light mode toggle button)
 ├── Route: '/' → Home.jsx (Header, About, Skills components)
 ├── Route: '/projects' → Projects.jsx (Dynamic project cards with category filter state)
 ├── Route: '/contact' → Contact.jsx (Controlled inputs, live character count, help tooltip
```

```
  toggle)
   ├── Route: '*' → NotFound.jsx (404 custom error component for undefined paths)
   └── Footer.jsx (Persistent footer with academic details and GitHub repo URL)
```

❖ Output Screenshots & Verification

1. Terminal Setup & React Router Installation (Windows Command Prompt - cmd.exe)

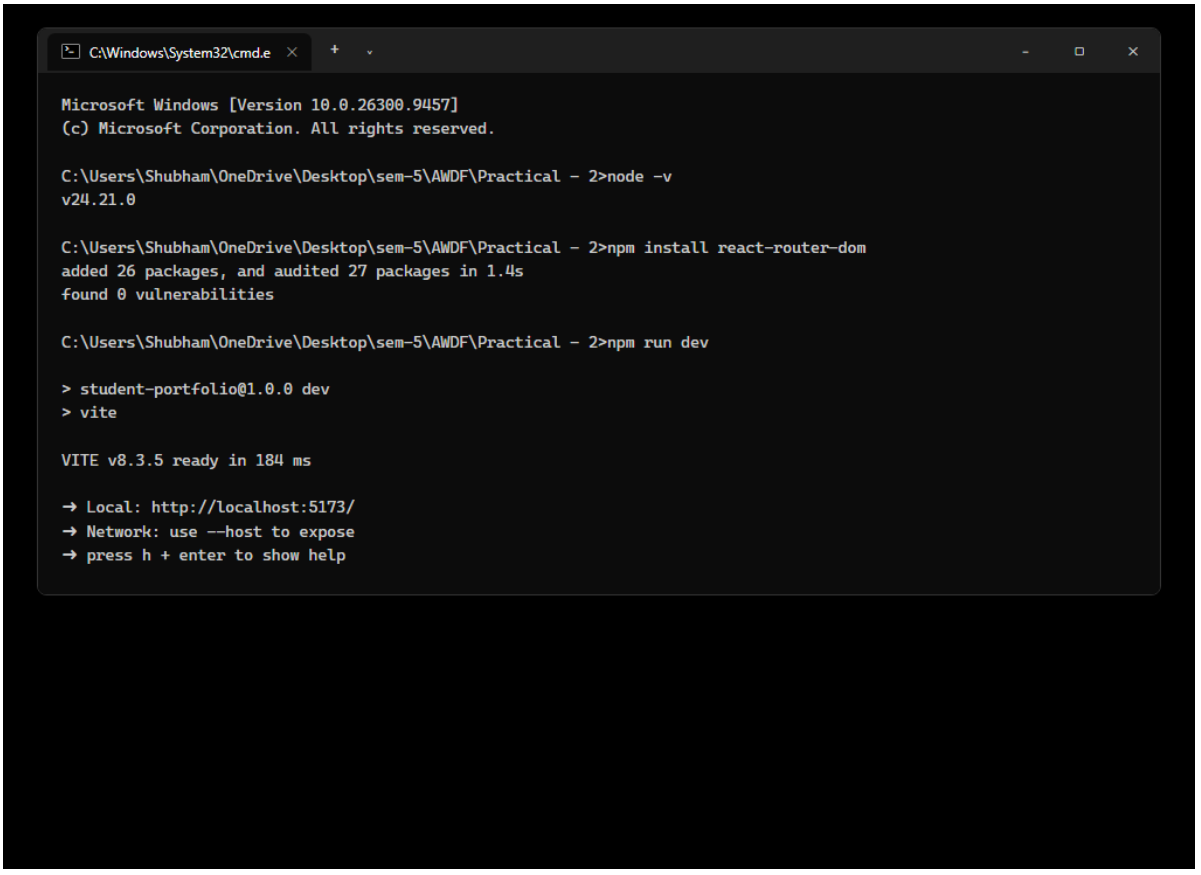


Figure: Execution of npm install react-router-dom, package verification, and starting the Vite development server on http://localhost:5173/ inside monochrome Windows Command Prompt (cmd.exe).

2. Home Route ('/') – Multi-Page Navigation & Hero Presentation

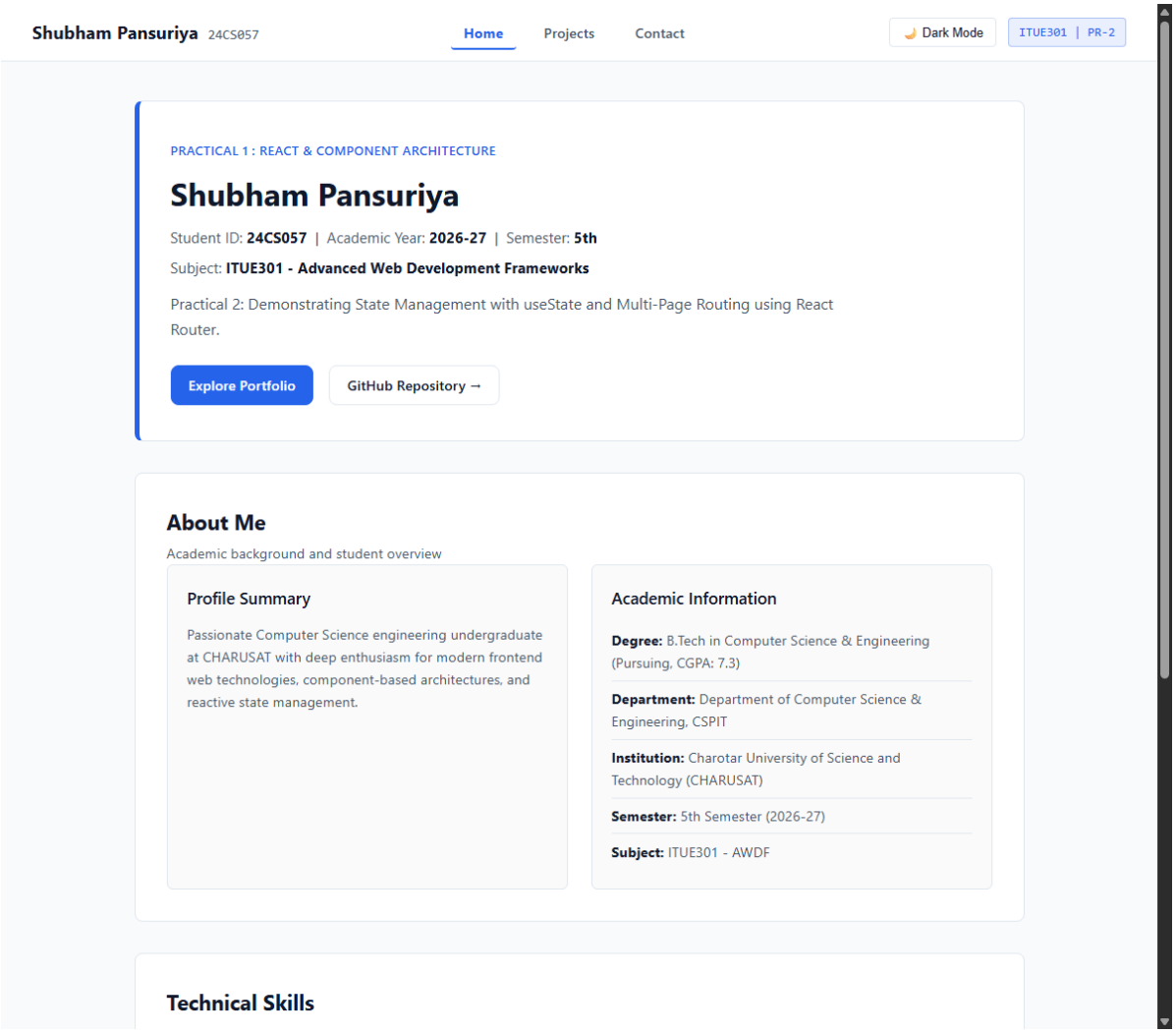


Figure: Rendering of the Home page route featuring client-side React Router navigation bar, hero presentation with student details, about section, and technical skills.

3. Projects Route ('/projects') – Dynamic Filter State

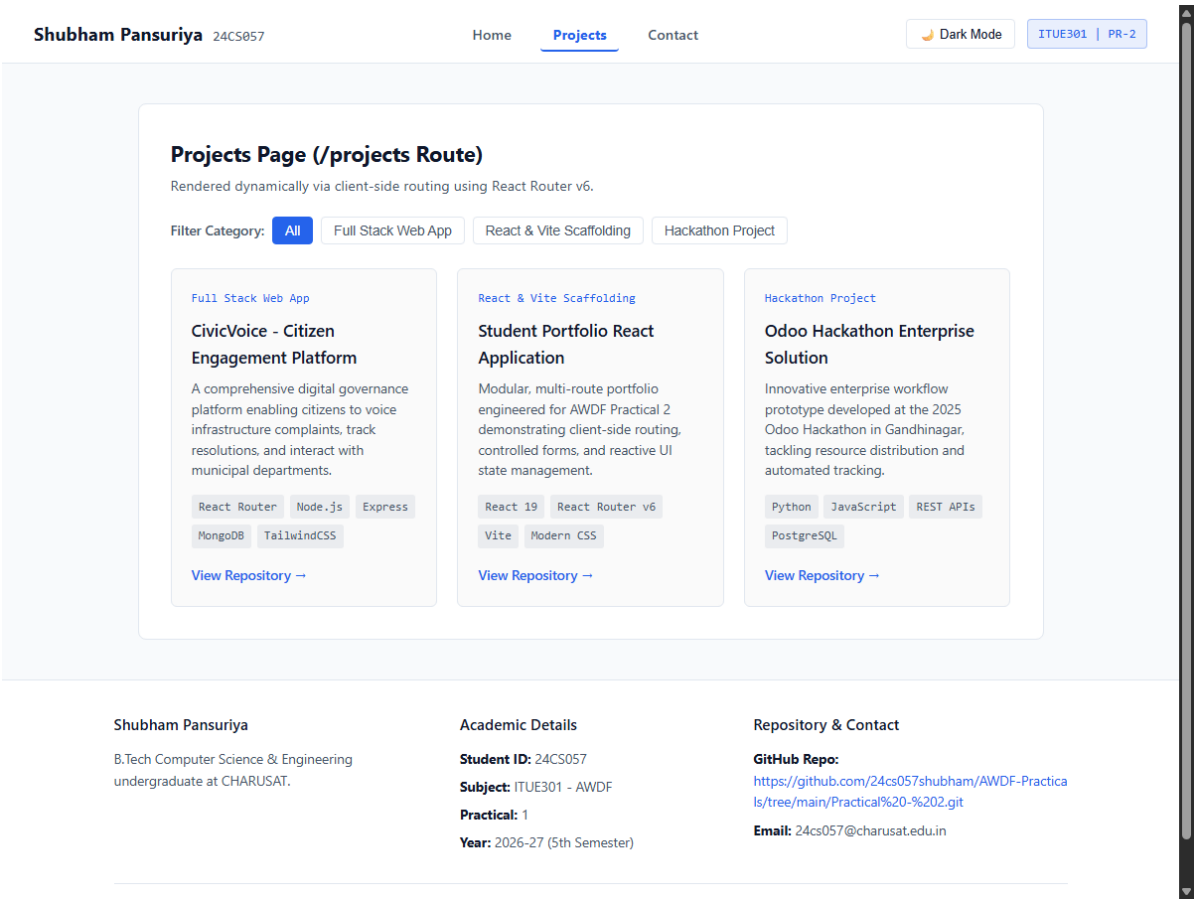


Figure: Dedicated Projects page rendered via client-side routing. Features category filtering state managed by useState without page reloads.

4. Contact Route ('/contact') – Controlled Inputs & Live Character Count

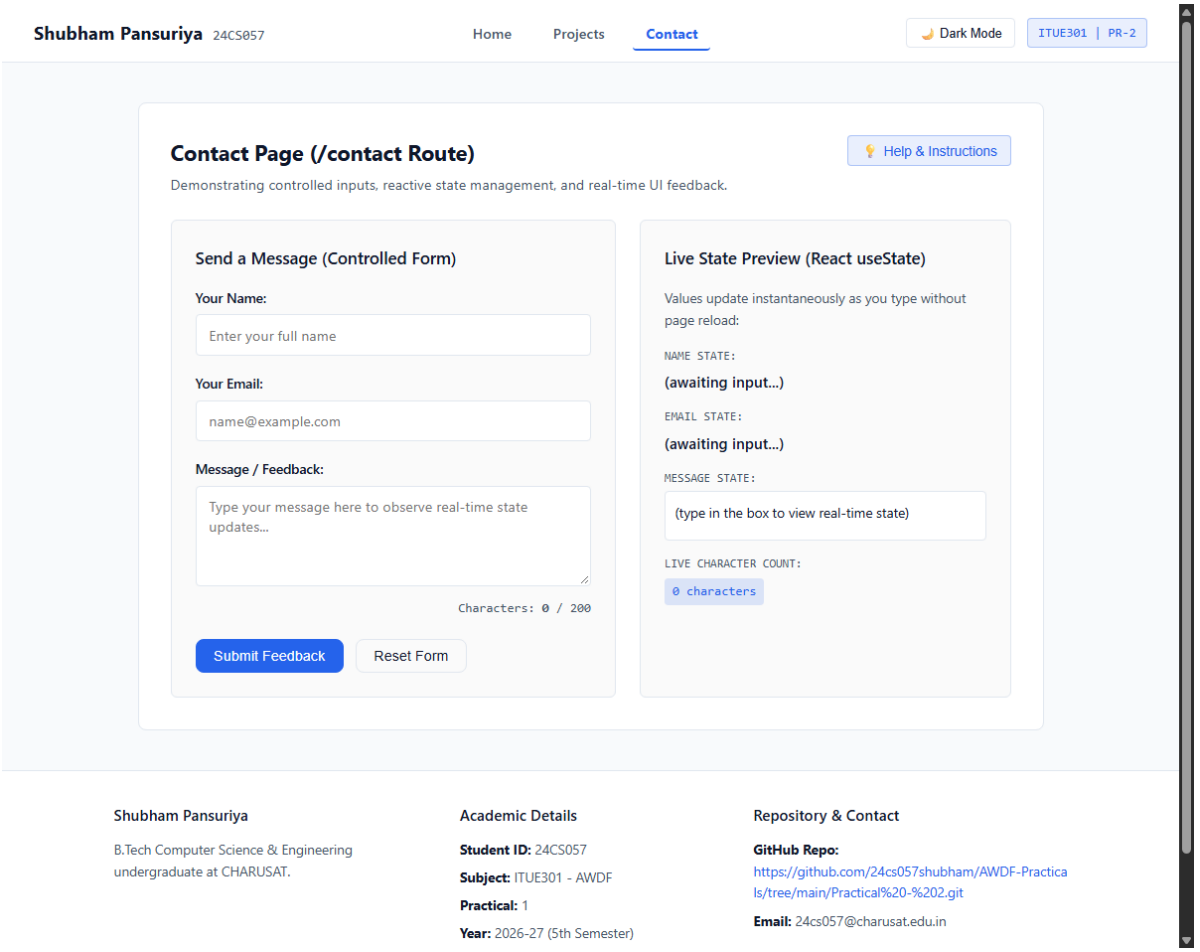


Figure: Contact page demonstrating controlled form inputs tied to useState, live character counting below the textarea, toggleable help box, and real-time preview.

5. 404 Not Found Route ('*') – Custom Error Handling

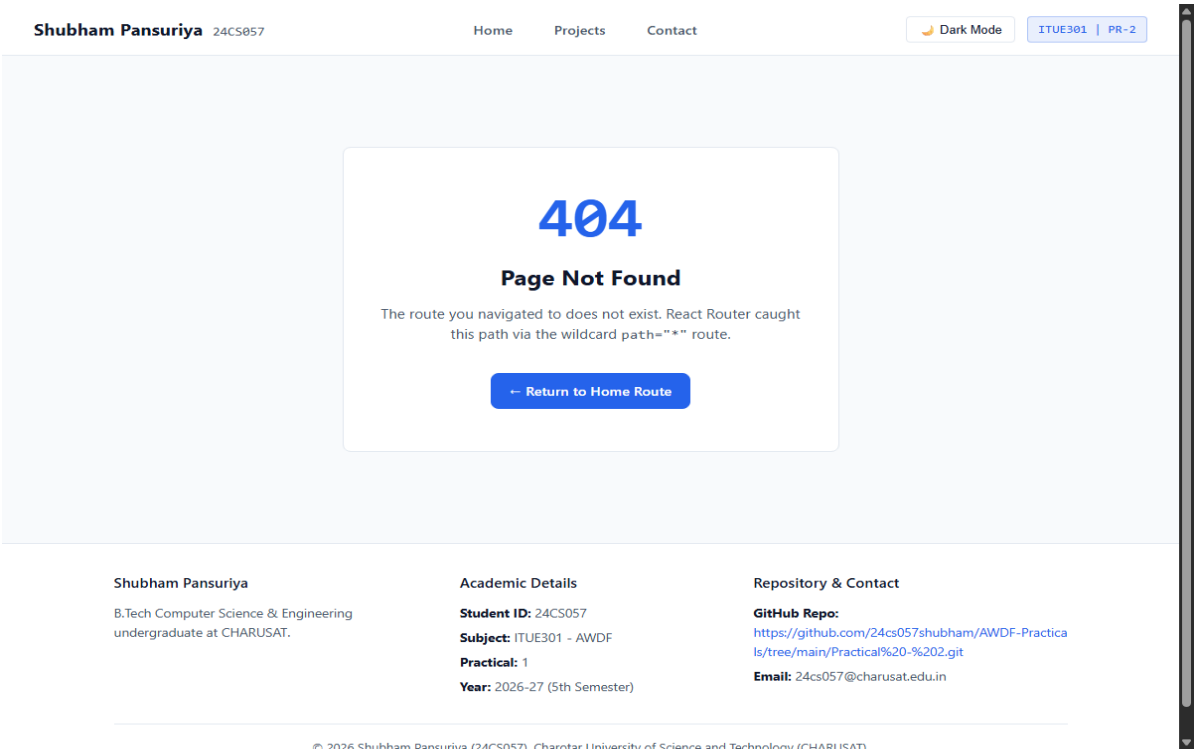


Figure: Custom 404 error page displayed when navigating to undefined URLs, featuring a clean error card and return navigation link.

6. React DevTools State & Route Inspection (Windows)

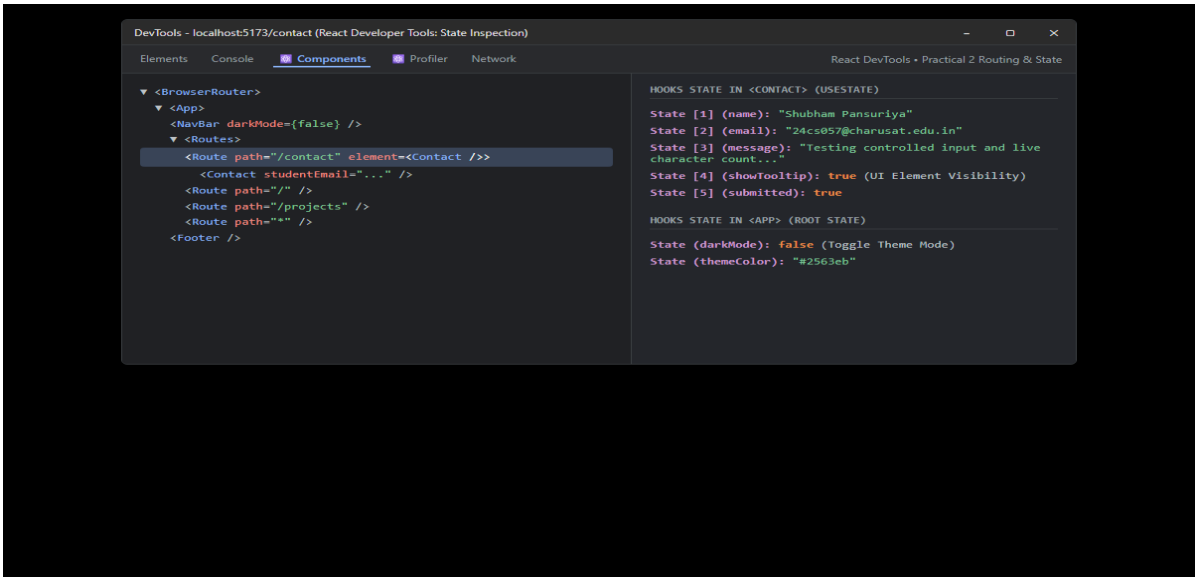


Figure: Inspection in React DevTools confirming the <BrowserRouter> and <Routes> hierarchy, and inspecting active useState hooks across Contact and App components.

7. GitHub Repository Evidence (AWDF-PR-2 on Windows)

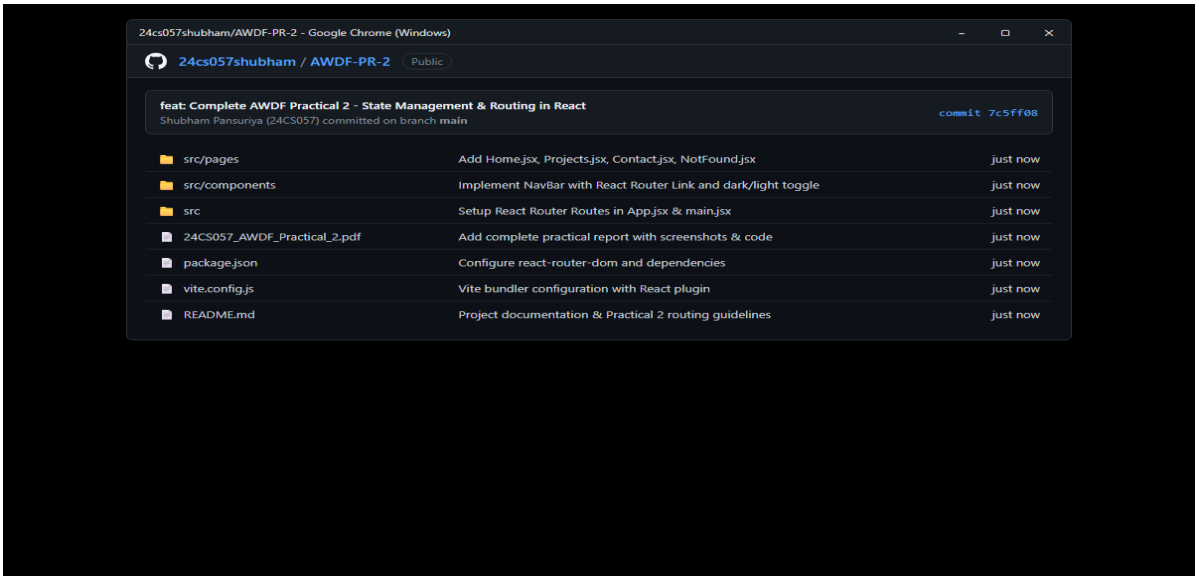


Figure: Public GitHub repository at https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%202 with working multi-route application, clean .gitignore, and commit history.

❖ Source Code Implementation

File: src/main.jsx (BrowserRouter Setup)

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

File: src/App.jsx (Route Definitions & Theme State)

```
import React, { useState } from 'react';
import { Routes, Route } from 'react-router-dom';
import NavBar from './components/NavBar.jsx';
import Footer from './components/Footer.jsx';
import Home from './pages/Home.jsx';
import Projects from './pages/Projects.jsx';
import Contact from './pages/Contact.jsx';
import NotFound from './pages/NotFound.jsx';
import './App.css';

function App() {
  // 1. Dark/Light Mode state (Supplementary Requirement)
  const [darkMode, setDarkMode] = useState(false);

  // 2. Theme accent color state
  const [themeColor, setThemeColor] = useState('#2563eb');

  const studentData = {
    name: "Shubham Pansuriya",
    studentId: "24CS057",
    courseCode: "ITUE301",
```

```
    courseTitle: "Advanced Web Development Frameworks",
    academicYear: "2026-27",
    semester: "5th Semester",
    institution: "Charotar University of Science and Technology (CHARUSAT)",
    department: "Department of Computer Science & Engineering, CSPIT",
    email: "24cs057@charusat.edu.in",
    githubRepo: "https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%202",
    bio: "Passionate Computer Science engineering undergraduate at CHARUSAT with deep enthusiasm
for modern frontend web technologies, component-based architectures, and reactive state
management.",
    education: "B.Tech in Computer Science & Engineering (Pursuing, CGPA: 7.3)"
  };

  const skills = [
    { name: "React.js", level: "Advanced" },
    { name: "React Router v6", level: "Proficient" },
    { name: "useState & Hooks", level: "Advanced" },
    { name: "JavaScript (ES6+)", level: "Advanced" },
    { name: "Vite & Build Tooling", level: "Proficient" },
    { name: "HTML5 & CSS3", level: "Expert" },
    { name: "Node.js & Express", level: "Proficient" },
    { name: "Git & GitHub", level: "Proficient" }
  ];

  const projects = [
    {
      id: 1,
      title: "CivicVoice - Citizen Engagement Platform",
      category: "Full Stack Web App",
      description: "A comprehensive digital governance platform enabling citizens to voice
infrastructure complaints, track resolutions, and interact with municipal departments.",
      tech: ["React Router", "Node.js", "Express", "MongoDB", "TailwindCSS"],
      link: "https://github.com/ShubhamPansuriya23"
    },
    {
      id: 2,
      title: "Student Portfolio React Application",
      category: "React & Vite Scaffolding",
      description: "Modular, multi-route portfolio engineered for AWDF Practical 2 demonstrating
client-side routing, controlled forms, and reactive UI state management.",
      tech: ["React 19", "React Router v6", "Vite", "Modern CSS"],
      link: "https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%202"
    },
    {
      id: 3,
      title: "Odoo Hackathon Enterprise Solution",
      category: "Hackathon Project",
      description: "Innovative enterprise workflow prototype developed at the 2025 Odoo Hackathon
in Gandhinagar, tackling resource distribution and automated tracking.",
      tech: ["Python", "JavaScript", "REST APIs", "PostgreSQL"],
      link: "https://github.com/ShubhamPansuriya23"
    }
  ];

  const toggleTheme = () => {
    setDarkMode(!darkMode);
  };

  return (
    <div className={`portfolio-app ${darkMode ? 'dark-mode' : 'light-mode'}`}>
      {/* Navigation Bar across all routes */}
      <NavBar
        darkMode={darkMode}
        onToggleTheme={toggleTheme}
      />
```

```
      {/* Main Routed Content */}
      <main className="main-content">
        <Routes>
          <Route
            path="/"
            element={<Home studentData={studentData} skills={skills} themeColor={themeColor} />}
          />
          <Route
            path="/projects"
            element={<Projects projectList={projects} />}
          />
          <Route
            path="/contact"
            element={<Contact studentEmail={studentData.email} />}
          />
          {/* Wildcard 404 Route */}
          <Route
            path="*"
            element={<NotFound />}
          />
        </Routes>
      </main>

      {/* Persistent Footer */}
      <Footer
        studentName={studentData.name}
        studentId={studentData.studentId}
        email={studentData.email}
        repoUrl={studentData.githubRepo}
      />
    </div>
  );
}

export default App;
```

File: src/components/NavBar.jsx (React Router NavLink & Toggle)

```
import React from 'react';
import { NavLink } from 'react-router-dom';

function NavBar({ darkMode, onToggleTheme }) {
  return (
    <nav className="navbar">
      <div className="nav-brand">
        <NavLink to="/" className="brand-link">
          <span className="brand-name">Shubham Pansuriya</span>
          <span className="brand-id">24CS057</span>
        </NavLink>
      </div>

      <ul className="nav-links">
        <li>
          <NavLink
            to="/"
            className={({ isActive }) => `nav-link-btn ${isActive ? 'active' : ''}`}
          >
            Home
          </NavLink>
        </li>
        <li>
          <NavLink
```

```
            to="/projects"
            className={({ isActive }) => `nav-link-btn ${isActive ? 'active' : ''}`}
          >
            Projects
          </NavLink>
        </li>
        <li>
          <NavLink
            to="/contact"
            className={({ isActive }) => `nav-link-btn ${isActive ? 'active' : ''}`}
          >
            Contact
          </NavLink>
        </li>
      </ul>

      <div className="nav-actions">
        <button
          onClick={onToggleTheme}
          className="theme-toggle-btn"
          title="Toggle Dark / Light Mode"
        >
          {darkMode ? '◌ Light Mode' : '🌙 Dark Mode'}
        </button>
        <span className="nav-badge">ITUE301 | PR-2</span>
      </div>
    </nav>
  );
}

export default NavBar;
```

File: src/pages/Home.jsx

```
import React from 'react';
import Header from '../components/Header.jsx';
import About from '../components/About.jsx';
import Skills from '../components/Skills.jsx';

function Home({ studentData, skills, themeColor }) {
  return (
    <div className="page-container">
      <Header
        name={studentData.name}
        studentId={studentData.studentId}
        courseTitle={`${studentData.courseCode} - ${studentData.courseTitle}`}
        subtitle="Practical 2: Demonstrating State Management with useState and Multi-Page Routing
using React Router."
        themeColor={themeColor}
      />

      <About
        bio={studentData.bio}
        education={studentData.education}
        department={studentData.department}
        institution={studentData.institution}
      />

      <Skills skillList={skills} />
    </div>
  );
}
```

```
export default Home;
```

File: src/pages/Projects.jsx (Route with Filter State)

```
import React, { useState } from 'react';

function Projects({ projectList }) {
  const [filter, setFilter] = useState('All');

  const categories = ['All', 'Full Stack Web App', 'React & Vite Scaffolding', 'Hackathon
Project'];

  const filteredProjects = filter === 'All'
    ? projectList
    : projectList.filter((p) => p.category === filter);

  return (
    <div className="page-container">
      <section className="portfolio-section">
        <h2 className="section-title">Projects Page (/projects Route)</h2>
        <p className="section-desc">
          Rendered dynamically via client-side routing using React Router v6.
        </p>

        {/* Category Filter buttons using useState */}
        <div className="filter-bar">
          <span className="filter-label">Filter Category:</span>
          {categories.map((cat) => (
            <button
              key={cat}
              className={`filter-btn ${filter === cat ? 'active' : ''}`}
              onClick={() => setFilter(cat)}
            >
              {cat}
            </button>
          ))}
        </div>

        <div className="projects-grid">
          {filteredProjects && filteredProjects.map((project, idx) => (
            <div key={project.id || idx} className="project-card">
              <span className="project-badge">{project.category}</span>
              <h3 className="project-title">{project.title}</h3>
              <p className="project-desc">{project.description}</p>
              <div className="project-tech">
                {project.tech && project.tech.map((t) => (
                  <span key={t} className="tech-tag">{t}</span>
                ))}
              </div>
              {project.link && (
                <a
                  href={project.link}
                  target="_blank"
                  rel="noopener noreferrer"
                  className="project-link"
                >
                  View Repository &rarr;
                </a>
              )}
            </div>
          ))}
        </div>
      </section>
```

```
    </div>
  );
}

export default Projects;
```

File: src/pages/Contact.jsx (Controlled Inputs & Character Count)

```
import React, { useState } from 'react';

function Contact({ studentEmail }) {
  // 1. Controlled form input states using useState
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [message, setMessage] = useState('');

  // 2. Second useState variable: Toggle visibility of help tooltip (Lab Requirement #6)
  const [showTooltip, setShowTooltip] = useState(false);

  // Submission feedback state
  const [submitted, setSubmitted] = useState(false);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (message.trim()) {
      setSubmitted(true);
    }
  };

  const handleReset = () => {
    setName('');
    setEmail('');
    setMessage('');
    setSubmitted(false);
  };

  return (
    <div className="page-container">
      <section className="portfolio-section">
        <div className="section-title-row">
          <div>
            <h2 className="section-title">Contact Page (/contact Route)</h2>
            <p className="section-desc">
              Demonstrating controlled inputs, reactive state management, and real-time UI
feedback.
            </p>
          </div>

          {/* Toggle Help Tooltip (useState UI visibility requirement) */}
          <button
            type="button"
            className="tooltip-toggle-btn"
            onClick={() => setShowTooltip(!showTooltip)}
          >
            {showTooltip ? '✕ Hide Help' : '💡 Help & Instructions'}
          </button>
        </div>

        {/* Toggled Help Box */}
        {showTooltip && (
          <div className="help-box">
            <strong>State Management Guide (Practical 2):</strong>
            <p>
```

```
              Every keystroke in the input fields triggers an <code>onChange</code> event handler,
updating the React component state via <code>useState()</code>. The input value is bound to state
(controlled component), enabling real-time live preview and instant character counting.
            </p>
          </div>
        )}

        <div className="contact-grid">
          {/* Controlled Form */}
          <div className="form-card">
            <h3>Send a Message (Controlled Form)</h3>
            <form onSubmit={handleSubmit}>
              <div className="form-group">
                <label htmlFor="name-input">Your Name:</label>
                <input
                  id="name-input"
                  type="text"
                  className="form-control"
                  placeholder="Enter your full name"
                  value={name}
                  onChange={(e) => setName(e.target.value)}
                  required
                />
              </div>

              <div className="form-group">
                <label htmlFor="email-input">Your Email:</label>
                <input
                  id="email-input"
                  type="email"
                  className="form-control"
                  placeholder="name@example.com"
                  value={email}
                  onChange={(e) => setEmail(e.target.value)}
                  required
                />
              </div>

              <div className="form-group">
                <label htmlFor="message-input">Message / Feedback:</label>
                <textarea
                  id="message-input"
                  rows="4"
                  className="form-control"
                  placeholder="Type your message here to observe real-time state updates..."
                  value={message}
                  onChange={(e) => setMessage(e.target.value)}
                  required
                />
                {/* Live Character Count (Supplementary Requirement) */}
                <div className="char-counter">
                  Characters: <strong>{message.length}</strong> / 200
                </div>
              </div>

              <div className="form-actions">
                <button type="submit" className="btn btn-primary">
                  Submit Feedback
                </button>
                <button type="button" onClick={handleReset} className="btn btn-secondary">
                  Reset Form
                </button>
              </div>
            </form>
```

```
            {submitted && (
              <div className="success-banner">
                ✓ State Successfully Captured! Thank you, <strong>{name}</strong>.
              </div>
            )}
          </div>

          {/* Real-Time State Preview Card */}
          <div className="preview-card">
            <h3>Live State Preview (React useState)</h3>
            <p className="preview-desc">
              Values update instantaneously as you type without page reload:
            </p>
            <div className="preview-item">
              <span className="preview-label">Name State:</span>
              <span className="preview-value">{name || '(awaiting input...)'}</span>
            </div>
            <div className="preview-item">
              <span className="preview-label">Email State:</span>
              <span className="preview-value">{email || '(awaiting input...)'}</span>
            </div>
            <div className="preview-item">
              <span className="preview-label">Message State:</span>
              <p className="preview-box">{message || '(type in the box to view real-time
state)'}</p>
            </div>
            <div className="preview-item">
              <span className="preview-label">Live Character Count:</span>
              <span className="badge-count">{message.length} characters</span>
            </div>
          </div>
        </div>
      </section>
    </div>
  );
}

export default Contact;
```

File: src/pages/NotFound.jsx (404 Custom Error Route)

```
import React from 'react';
import { Link } from 'react-router-dom';

function NotFound() {
  return (
    <div className="page-container notfound-container">
      <div className="notfound-card">
        <span className="notfound-code">404</span>
        <h2 className="notfound-title">Page Not Found</h2>
        <p className="notfound-desc">
          The route you navigated to does not exist. React Router caught this path via the
wildcard <code>path="*"</code> route.
        </p>
        <Link to="/" className="btn btn-primary">
          &larr; Return to Home Route
        </Link>
      </div>
    </div>
  );
}
```

```
export default NotFound;
```

File: src/App.css (Excerpt)

```
/* App Layout */
.portfolio-app {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  transition: background-color 0.25s ease, color 0.25s ease;
}

/* Light / Dark Mode Tokens */
.light-mode {
  --bg-color: #f8fafc;
  --card-bg: #ffffff;
  --text-main: #0f172a;
  --text-muted: #475569;
  --border-color: #e2e8f0;
  --input-bg: #ffffff;
}

.dark-mode {
  --bg-color: #0f172a;
  --card-bg: #1e293b;
  --text-main: #f8fafc;
  --text-muted: #94a3b8;
  --border-color: #334155;
  --input-bg: #0f172a;
}

body {
  background-color: var(--bg-color);
  color: var(--text-main);
}

.portfolio-app {
  background-color: var(--bg-color);
  color: var(--text-main);
}

/* Navbar */
.navbar {
  position: sticky;
  top: 0;
  z-index: 50;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0.85rem 2rem;
  background-color: var(--card-bg);
  border-bottom: 1px solid var(--border-color);
}

.nav-brand .brand-link {
  display: flex;
  align-items: baseline;
  gap: 0.6rem;
  text-decoration: none;
}

.brand-name {
  font-size: 1.15rem;
```

```
  font-weight: 700;
  color: var(--text-main);
}

.brand-id {
  font-size: 0.85rem;
  font-family: var(--font-mono);
  color: var(--text-muted);
}

.nav-links {
  display: flex;
  list-style: none;
  gap: 1rem;
}

.nav-link-btn {
  font-size: 0.9rem;
  font-weight: 500;
  color: var(--text-muted);
  text-decoration: none;
  padding: 0.4rem 0.8rem;
  border-radius: 4px;
  transition: all 0.2s;
}

.nav-link-btn:hover {
  color
/* ... full styles in repository ... */
```

❖ Key Questions & Analysis

Q1: What is the role of each component in the overall UI structure?

In this multi-page React architecture:
• BrowserRouter & Routes (in main.jsx and App.jsx): Wraps the component tree and synchronizes UI with the current browser URL path.
• NavBar: Serves as the global top bar using React Router's <NavLink> to switch views instantly without triggering browser page reloads.
• Home (/): Default landing route showcasing student overview, academic credentials, and core competencies.
• Projects (/projects): Route for displaying featured projects with category filtering state.
• Contact (/contact): Interactive route implementing controlled inputs, live character count, real-time preview, and tooltip visibility toggle.
• NotFound (*): Catch-all error route that intercepts unmatched paths and directs users back to safety.

Q2: How does React re-render when props or state change?

When a state updater function (e.g., setMessage(e.target.value)) is invoked, React schedules an asynchronous re-render of that component. During re-rendering, React executes the component function again, produces a new Virtual DOM tree, and diffs it against the prior tree. Only the precise real DOM nodes that reflect changes (such as the character counter and preview box) are mutated in the browser, making updates lightning fast without reloading the webpage.

Q3: Why is component reusability and client-side routing important in large-scale applications?

1. SPA Performance: Client-side routing eliminates heavy server roundtrips and full HTML document reloads, providing instant page transitions.
2. State Preservation: Application state (such as user session, dark mode, or shopping carts) remains preserved during page transitions.
3. Modular Maintainability: Dividing pages into dedicated route components (Home, Projects, Contact) allows isolated testing, lazy loading, and parallel feature development.